

WeatherGuard Attendance

Team: 04

Team members:

Chima Ohaechesi, Joshua Clemens, Judah Fisher, Roger Karam

Software Design Specification & Project Delivery Report

Version: v1.0.0

Date: (05/09/26)

Table of Contents

1 Introduction.....	3
2 System Design.....	4
2.1 High-Level Architecture.....	4
2.2 Design Alternatives Considered.....	4
2.3 Pattern 1 Observer.....	5
2.4 Pattern 2 State.....	6
2.5 Pattern 3 Strategy.....	7
2.6 Continuous Quality Improvement.....	8
2.7 Mapping to User Requirements.....	8
3 System Implementation.....	8
3.1 Observer in Action.....	8
3.2 State in Action.....	9
3.3 Strategy in Action.....	11
3.4 End-of-Session Summary.....	12
3.5 Code Quality and Convention.....	12
4 Utilization of External Learning Resources.....	12
4.1 Course Material.....	12
4.2 Books and Articles.....	13
4.3 Online Documentation.....	13
4.4 SE 370 Codebase.....	13
5 Lessons Learned.....	13
5.1 Josh Clemens - Integration and Code Review.....	13
5.2 Judah Fisher - Strategy.....	14
5.3 Chima Ohaechesi - Observer.....	14
5.4 Roger Karam - State.....	14

1 Introduction

WeatherGuard Attendance is a JavaFX desktop application that modernizes classroom attendance tracking by combining QR-code check-ins with real-time weather integration. The project began in SE 370 (Fall 2025) as a roster-and-session management tool; in SE 471 we kept the user-facing functionality intact and refactored the architecture so that it could scale to new behavior without further conditional sprawl. The deployment target is educational institutions in extreme-weather regions where attendance decisions must take environmental conditions into account.

Background. Manual roll call is slow and error prone; check-ins via QR are reportedly 80% faster. The SE 370 prototype already had a working JavaFX UI, MongoDB persistence, OpenWeatherMap integration, and a Netlify-hosted student check-in page. What it did not have was a clean way to evolve: attendance status was a hardcoded string, session lifecycle was a boolean, and the dashboard refreshed by polling the database every two seconds. The SE 471 refactor replaces these with three behavioral patterns drawn from Lecture 16 (Mediator · State · Strategy · Null) and the Team 6 Observer presentation, plus the Decorator pattern wrapping one of the Strategies.

Key features delivered this semester. Three required design patterns wired into production code paths: (1) **Observer** broadcasts every check-in as an immutable AttendanceEvent to any registered listener; (2) **State** models the session lifecycle as polymorphic Inactive · Active · Closed classes with onEnter / onExit hooks; (3) **Strategy** applies twice, an AttendanceRuleStrategyIF with strict, grace-period, and weather-lenieny variants, and a TemperatureDisplayStrategyIF with Fahrenheit and Celsius variants. A **Decorator** (WeatherLenienyStrategy) wraps the attendance rule to extend grace windows in extreme cold without modifying the inner Strategy. The original SE 370 patterns, Singleton on DatabaseManager, Façade on WeatherService, Transfer Object across the model layer, are preserved and integrated cleanly with the new behavioral layer.

Potential users. The target users are K-12 and university instructors in cold-weather or storm-prone regions, administrators who need attendance records correlated with operational decisions (delayed starts, cancellations), and students who want a frictionless check-in experience that does not depend on manual roll call. The architecture is intentionally extensible: a new attendance policy is one new Strategy class, a new dashboard or analytics consumer is one new Observer, and a new lifecycle phase (such as a grace period state) is one new State subclass. None of these additions touches existing controllers.

2 System Design

This section presents the high-level architecture, the design alternatives considered, and the justification for each pattern adopted.

2.1 High-Level Architecture

WeatherGuard follows a layered architecture inherited from SE 370 and preserved through the refactor. The Presentation layer is JavaFX with FXML controllers (AdminViewController, TeacherViewController, FiveDayForecastController). The Domain / Pattern layer is new this semester: it contains SessionState hierarchy, AttendanceRuleStrategyIF hierarchy, TemperatureDisplayStrategyIF hierarchy, and the Observer interfaces. The Integration layer wraps external services: WeatherService is a Façade over Weather, Forecast, and StaticMap; DatabaseManager is a Singleton over the MongoDB Atlas driver. The Data layer is a small set of Transfer Object models (Attendance, Session, Student, ClassInfo). A separate Web Portal tier hosted on Netlify accepts student check-ins and writes them into MongoDB as pending rows; the desktop app then applies its Strategy to upgrade them to present / late / absent.

2.2 Design Alternatives Considered

Three alternatives were evaluated against each weakness in the SE 370 codebase before the final pattern set was chosen. They are summarized in the table below; each row's rightmost column is the option implemented.

Concern	Option A	Option B	Adopted
Live attendance dashboard refresh	Keep 2-second JavaFX Timeline polling MongoDB	Migrate to MongoDB change streams (network-bound)	Observer over a Netlify-insert bridge. No UI polling, fan-out to multiple listeners
Session lifecycle	Keep boolean sessionActive + scattered guards	Enum + switch statement	State hierarchy with onEnter / onExit hooks. Atomic transitions, polymorphic dispatch
Attendance rule	Keep hardcoded "present" string	Configuration table read at startup	Strategy + Decorator. Runtime swap, weather-aware extensions without modifying inner rule
Temperature unit	Keep boolean useFahrenheit	Two display methods on WeatherService	Strategy inside the existing Façade. Single source of truth, zero controller change

2.3 Pattern 1 Observer

Implementation: `com.weatherboys.weatherguard.Observer`.

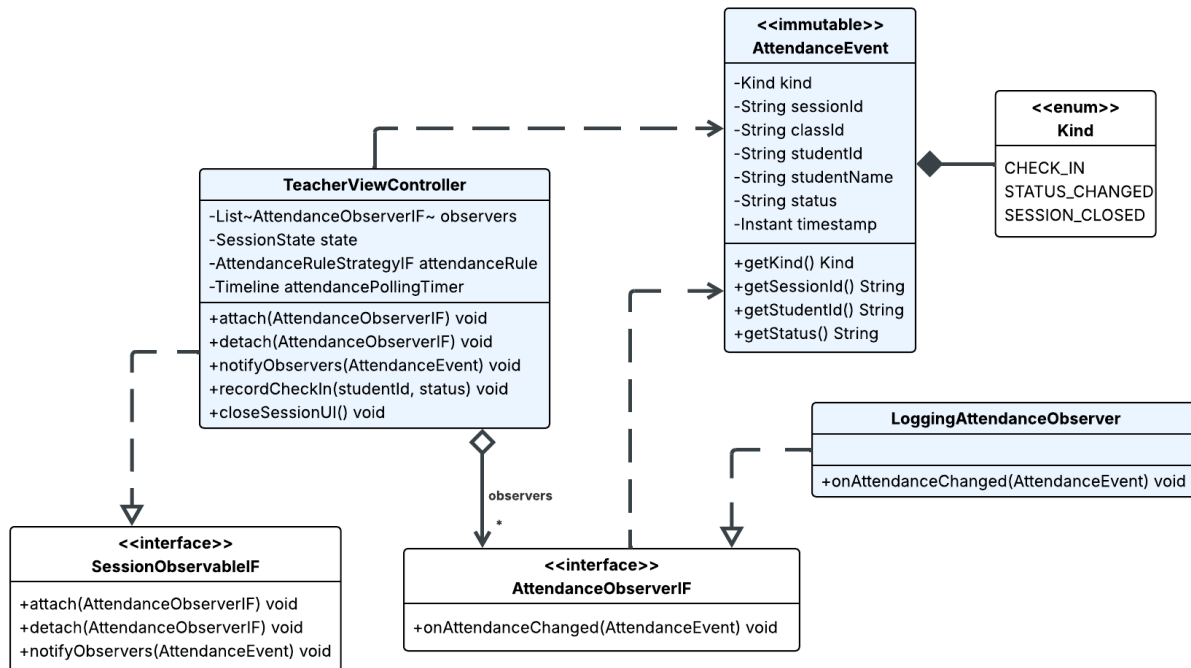


Figure 1. Observer pattern as implemented: SessionObservableIF on TeacherViewController, AttendanceObserverIF consumed by LoggingAttendanceObserver, immutable AttendanceEvent payload.

Justification. The most pressing architectural problem in the SE 370 codebase was that the Teacher dashboard relied on a JavaFX Timeline polling MongoDB every two seconds to detect new check-ins. This was brittle (interval-bound, network-dependent), wasted reads when no student had checked in, and was difficult to test. The Observer pattern defines a one-to-many dependency where a Subject notifies all registered Observers automatically whenever its state changes, eliminating polling at the consumer side.

Why not change streams? MongoDB change streams would still be a network watcher, just one layer down, and would couple the UI to a database feature. A message bus is overkill for a single-process desktop app. Observer keeps the dependency in-process, leaves DatabaseManager as the sole writer of attendance rows, and lets any number of consumers (the dashboard label painter, a console logger, a future analytics pipeline) plug into the same event stream without coordinating with each other.

Implementation note. The Subject role lives on TeacherViewController (which owns the active session and is the natural fan-out point). The first concrete Observer is LoggingAttendanceObserver; the dashboard label painter is the second. AttendanceEvent is a final immutable value object carrying kind, sessionId, classId, studentId, studentName, status, timestamp. The status field is set by the Strategy layer, so the Observer never needs to know how presence was decided.

2.4 Pattern 2 State

Implementation: `com.weatherboys.weatherguard.State`.

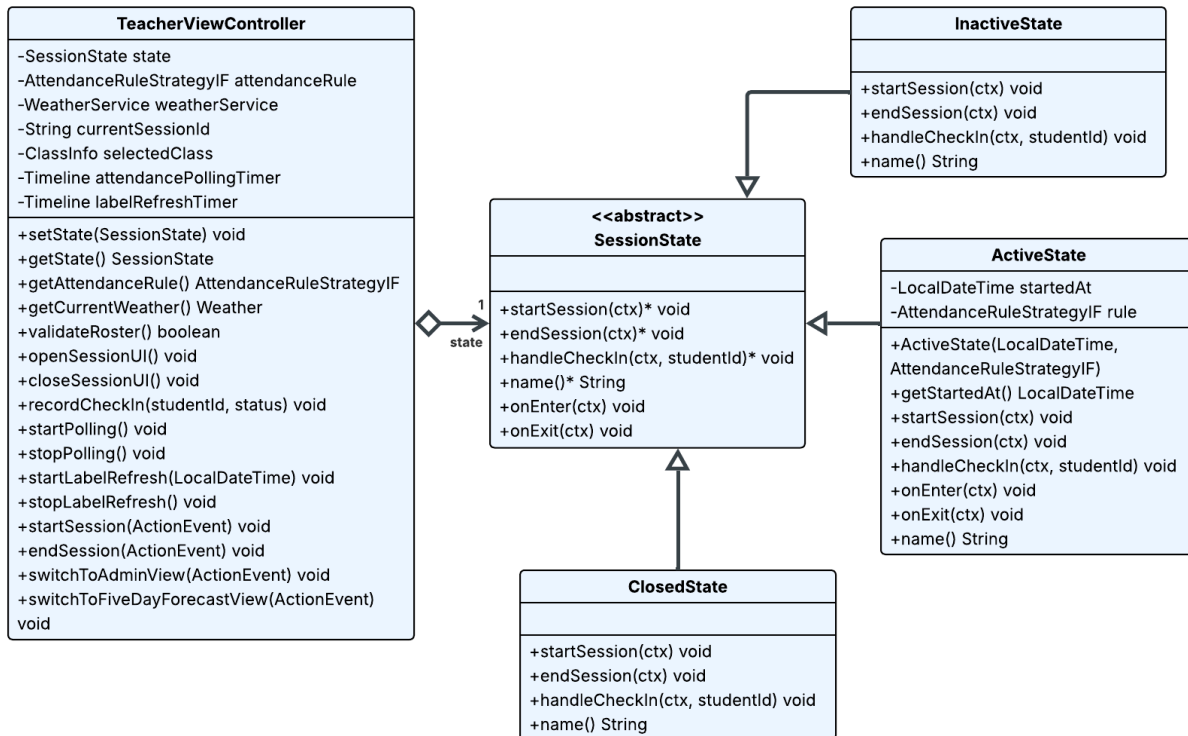


Figure 2. State pattern: SessionState abstract base, three concrete states, and the per-state transitions for startSession, endSession, and handleCheckIn.

Justification. An attendance session passes through three distinct lifecycle phases, each with its own rules: Inactive (no session running, Start enabled, End disabled, QR hidden), Active (session running, check-ins accepted, End enabled, QR shown), and Closed (session ended, controls locked, history viewable). The SE 370 codebase encoded this as boolean `sessionActive` with `if (sessionActive) ...` guards scattered across `TeacherViewController`, primitive obsession in textbook form. Adding a new phase (for example, a grace-period state where late check-ins are accepted but flagged) would require auditing every guard.

Why not an enum + switch? An enum centralizes the value but leaves behavior dispatch in switch statements, repeating the same conditional sprawl. State replaces that with polymorphism: each subclass owns its hooks and can carry its own data (e.g., `ActiveState` owns `startedAt` and the attendance rule, so the rule is locked in for the session's lifetime and cannot be corrupted by a mid-session swap).

Implementation note. Click handlers are reduced to one-liners: `startSession(ActionEvent)` calls `state.startSession(this)`; `endSession(ActionEvent)` calls `state.endSession(this)`. Transitions are atomic because `setState(SessionState)` runs the old state's `onExit` then the new state's `onEnter` in lockstep. Navigation handlers (`switchToAdminView`, `switchToFiveDayForecastView`) also delegate to `state.endSession(this)`, which closed an orphan-session bug that was invisible in the SE 370 codebase.

2.5 Pattern 3 Strategy

Implementation: com.weatherboys.weatherguard.Strategy.

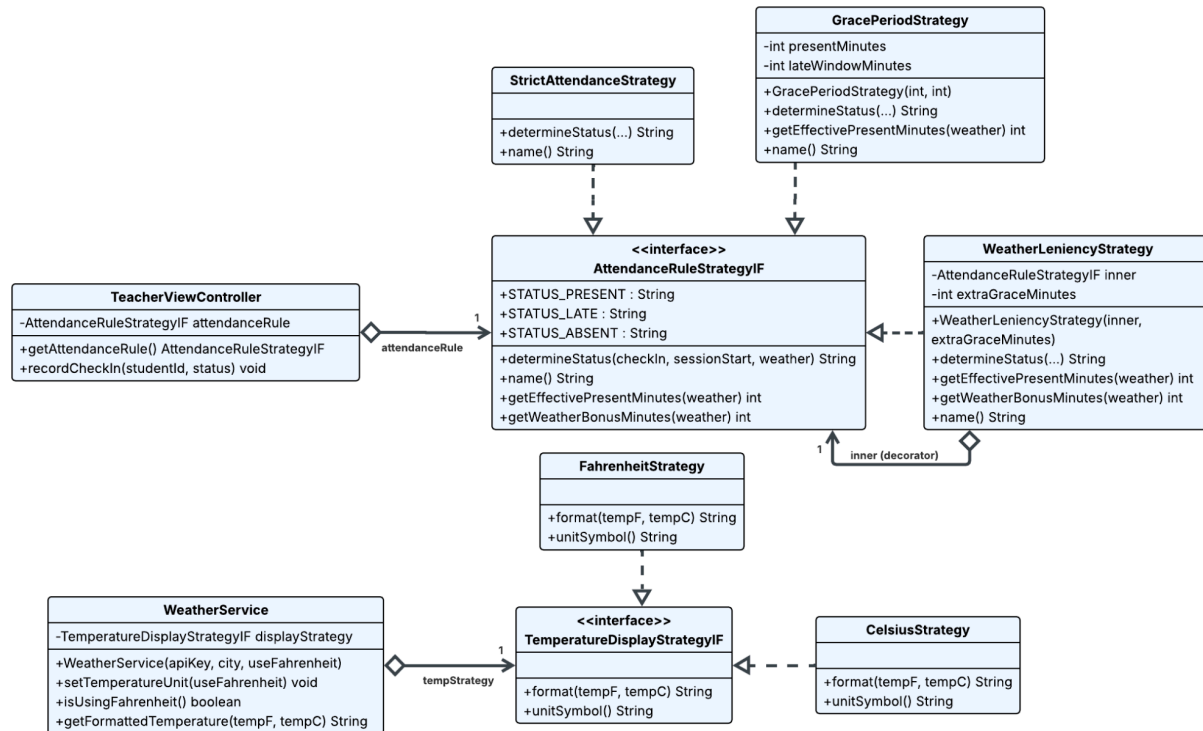


Figure 3. Strategy applied twice in WeatherGuard: AttendanceRuleStrategyIF with three concrete rules plus WeatherLeniencyStrategy as a Decorator, and TemperatureDisplayStrategyIF with Fahrenheit and Celsius variants held inside the WeatherService Façade.

Justification. WeatherGuard serves institutions in extreme-weather regions, which means the rules for what constitutes acceptable attendance vary by event and by instructor preference. Hardcoding "present" in DatabaseManager.markAttendance and boolean useFahrenheit in WeatherService made every policy change a code edit. Strategy defines a family of interchangeable algorithms behind a single interface, encapsulates each one, and lets the caller swap them at runtime without changing the context.

Two applications, one pattern. AttendanceRuleStrategyIF has three concrete implementations, StrictAttendanceStrategy (no grace), GracePeriodStrategy (configurable present and late windows), and WeatherLeniencyStrategy (a Decorator that wraps any inner rule and shifts the check-in time earlier when outdoor temperature drops below 32°F). TemperatureDisplayStrategyIF has two, FahrenheitStrategy and CelsiusStrategy, held inside WeatherService, so the existing Façade is the single source of truth for temperature display and the formerly duplicated if/else in three controllers is consolidated into one polymorphic call.

Decorator. The WeatherLeniencyStrategy is a true Decorator, it always delegates to the inner Strategy first and only modifies the result afterwards. An earlier draft short-circuited in extreme weather (returning present without ever calling the inner rule); that was substitution disguised as decoration. The corrected version asks the inner rule for its decision, and if it returns late, checks whether shifting the check-in time earlier by extraGraceMinutes would have produced a present. If yes, the cold-weather

grace covers the gap and the result is upgraded; otherwise the inner's decision passes through unchanged. This is the textbook semantic of Decorator: delegate first, modify after.

2.6 Continuous Quality Improvement

The architecture evolved across four integration phases. Each phase added one pattern, removed the corresponding code smell, and exposed a latent bug that the new pattern made visible. The pattern is consistent and worth naming explicitly: patterns as observability surfaces, externalized state transitions, broadcast events, and named Strategy outputs make inconsistencies elsewhere in the codebase visible by inspection.

Phase 1 Strategy (temperature). useFahrenheit deleted from WeatherService; FahrenheitStrategy / CelsiusStrategy installed inside the Façade. Surfaced and fixed: a transient Fahrenheit flicker on navigation back from Five-Day Forecast, caused by a default-then-patch sequence. Fix: 3-arg constructor that refuses to exist in a misconfigured state.

Phase 2 State (session lifecycle). sessionActive deleted; Inactive / Active / Closed hierarchy installed. Surfaced and fixed: orphan database sessions on Admin and Forecast navigation. Fix: navigation handlers also delegate to state.endSession(this).

Phase 3 Observer (attendance fan-out). Polling removed from TeacherViewController; AttendanceEvent broadcast on every check-in. Surfaced and fixed: duplicate CHECK_IN events from the polling-bridge loop (Observer's per-event log made it visible).

Phase 4 Strategy (attendance rule) + Decorator. "present" literal removed from DatabaseManager; GracePeriodStrategy wrapped by WeatherLeniencyStrategy (Decorator) installed in ActiveState. Surfaced and fixed: stale-vocabulary count in the closing pie chart, plus the WeatherLeniencyStrategy substitution-vs-decoration architectural error noted above.

2.7 Mapping to User Requirements

Every original SE 370 user requirement is preserved in the SE 471 implementation. QR-based check-in still works through the Netlify page; the Teacher View still shows the live roster with color-coded labels; OpenWeatherMap data still drives the current-conditions and five-day-forecast panels; CSV roster import still functions in the Admin View. The new pattern layer is additive, it changes how the system decides things, not what the user sees.

3 System Implementation

All three required patterns are wired into code paths in Code/WeatherGuard/. The screenshots below were captured during a live demo session at CS671 / McMurdo (-18°F) with nine check-ins; they show the patterns firing in concert and the bugs they helped surface.

3.1 Observer in Action

TeacherViewController registers a LoggingAttendanceObserver during onEnter of ActiveState and detaches it during onExit. Every check-in fires an AttendanceEvent with the Strategy-decided status. The console output below was captured from a live session and shows three concrete behaviors: (a) the State transition Inactive to Active, (b) the Strategy/Decorator decision for student S017 (boundary case where

the Decorator upgrades late to present), and (c) the Observer broadcast carrying that status.

```

1  public void recordCheckIn(String id, String status) {
2      studentStatusMap.put(id, status);
3      AttendanceEvent e = new AttendanceEvent( Kind.CHECK_IN,
4          currentSessionId, selectedClass.getClassId(), id, null,
5          status, LocalDateTime.now());
6      notifyObservers(e); // fan out to all subscribers
7  }
8  // initialize() - one line wires the first subscriber:
9  attach(new LoggingAttendanceObserver());

```

Figure 4. SessionObservableIF implementation in TeacherViewController, attach, detach, and notifyObservers with per-observer exception swallowing.

```

[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S021 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S021 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S017 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S017 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S033 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S033 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S027 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S027 status=present

```

Figure 5. Live console output during a check-in. [State] tag from the State pattern; [AttendanceRule] tag from the Strategy/Decorator chain; [Observer] broadcast with final status.

3.2 State in Action

SessionState hooks onEnter and onExit orchestrate every UI transition. Click handlers are one-line delegations. The controller never inspects the state via instanceof; behavior is dispatched polymorphically. The screenshots below show the controller-side delegation pattern and a per-transition log line.

```

1  public void setState(SessionState s) {
2      state.onExit(this);
3      this.state = s;
4      state.onEnter(this);
5  }
6  // ActiveState.handleCheckIn - owns the rule
7  public void handleCheckIn(ctx, id) {
8      String status = rule.determineStatus(
9          LocalDateTime.now(), startedAt,
10         ctx.getCurrentWeather());
11         ctx.recordCheckIn(id, status);
12     }

```

Figure 6. One-line click handlers, state.startSession(this) and state.endSession(this), replacing the SE 370 if (sessionActive) sprawl.

Admin View

Late!

Advanced AI

CS671

Professor: Mr. Anderson

End Session

Students:

Agent Brown, S013	Niobe, S021
Agent Jones, S012	Oracle, S020
Agent Smith, S011	Rachael Tyrell, S025
Artoo Detoo, S037	Rick Deckard, S039
Ava, S026	Roy Batty, S024
Commander Lock, S022	Sarah Connor, S029
Cypher, S016	See Threepio, S038
Data, S027	Seraph, S023
Dozer, S018	Sonny, S033
EVE, S031	T-800, S028
Ellen Ripley, S040	Tank, S017

Figure 7. Live UI showing the Active state, Start disabled, End enabled, QR code visible, weather data displayed. The previous state's onExit guarantees this configuration is atomic.

3.3 Strategy in Action

The attendance Strategy is composed at construction time on `ActiveState`: `new WeatherLeniencyStrategy(new GracePeriodStrategy())`. The temperature Strategy lives inside `WeatherService` and is swapped at runtime when the user toggles the F/C button. The figures below show the strategy implementation, the runtime label name (`WeatherLenient(+1min in cold, on Grace(1/10))`) which is self-documenting because the layering reads directly off the log, and four boundary screenshots from the demo session that demonstrate present, late, absent, and the cold-weather upgrade.

```

1  // WeatherService (the Facade) holds the unit:
2  private TemperatureDisplayStrategyIF tempStrategy;
3  public void setTemperatureUnit(boolean useF) {
4      tempStrategy = useF ? new FahrenheitStrategy(): new
        CelsiusStrategy();
5  }
6  // ActiveState carries the attendance rule:
7  private final AttendanceRuleStrategyIF rule;
8  // Decorator wraps any rule – same interface:
9  rule = new WeatherLeniencyStrategy(
10     new GracePeriodStrategy(1, 10));

```

Figure 8. `WeatherLeniencyStrategy.determineStatus` delegates to the inner `AttendanceRuleStrategyIF` first, then asks whether shifting the check-in time earlier by `extraGraceMinutes` would have produced a better status. True Decorator semantics, delegate first, modify after.



Figure 9. Four boundary outcomes from the Strategy/Decorator chain in the demo session: present, late, absent, and a cold-weather grace upgrade.

3.4 End-of-Session Summary

When `state.endSession(this)` fires, the controller's `closeSessionUI` aggregates the per-student `studentStatusMap` into a three-slice pie chart. The slice vocabulary matches the Strategy's output exactly, a deliberate decision so that an inconsistency between the Strategy and the chart is immediately visible by name.

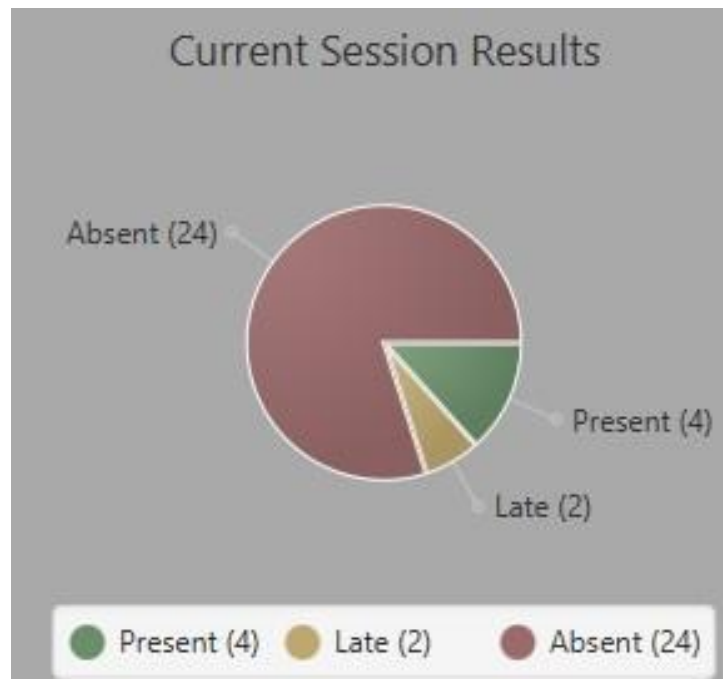


Figure 10. End-of-session pie chart. Present (5), Late (4), Absent (21). Slice colors match the per-student label colors used during the live session.

3.5 Code Quality and Convention

The team adopted Java standard naming conventions and a four-package layout (State, Strategy, Observer, Weather) so that each pattern lives in its own namespace and is discoverable by package name alone. Pattern-bearing interfaces use the IF suffix (`SessionObservableIF`, `AttendanceObserverIF`, `AttendanceRuleStrategyIF`, `TemperatureDisplayStrategyIF`) so the role is obvious at the import site. JUnit tests cover the Strategy boundary cases (grace-period transitions, weather lenience in cold versus mild conditions, display-strategy swap). Code review happened on every PR, we reviewed each teammate's pattern PR before merging.

4 Utilization of External Learning Resources

The team relied on the SE 471 lecture material as the primary source of truth for pattern structure and intent, and supplemented it with a small number of well-scoped external references for implementation details specific to the Java + JavaFX + MongoDB stack. References are listed by category below.

4.1 Course Material

Lecture 16 Mediator, State, Strategy, Null. Primary source for State, Strategy, and the Decorator-vs-Strategy distinction discussed during in-class examples. Cited directly in pattern justifications above. **Lecture 12 Decorator, Dynamic Linkage.** Source for the Decorator semantics applied to `WeatherLeniencyStrategy`. **Team 6 Flipped-Classroom presentation Observer.** Primary source for the

Observer structure used throughout the codebase, including the attach / detach / notifyObservers idiom and the memory-leak warning on observer cleanup. **Chain of Responsibility + Composite**. Considered for the bonus weather-escalation feature; The textbook example (physical-security alarm hierarchy) is cited in the proposal as the reference design.

4.2 Books and Articles

Gamma, Helm, Johnson, Vlissides Design Patterns: Elements of Reusable Object-Oriented Software (Addison-Wesley, 1995). Used as the canonical reference for Observer, State, Strategy, and Decorator semantics. The course textbook Lecture 16 derives its diagrams from this source; we cross-checked our implementation against the GoF text directly when the Decorator-vs-Strategy distinction first came up in code review. Freeman & Robson Head First Design Patterns (O'Reilly, 2nd ed., 2020). Consulted for accessible explanations when teammates were learning a pattern for the first time. The duck-behavior example (Strategy) and the gumball-machine example (State) were referenced.

4.3 Online Documentation

Oracle Java SE Documentation (docs.oracle.com/en/java/). `java.util.concurrent.CopyOnWriteArrayList` for the observer list, `java.time.LocalDateTime` arithmetic for the Strategy boundary tests. **OpenJFX Documentation** (openjfx.io/javadoc/21/). `Platform.runLater` for marshalling Observer notifications onto the JavaFX application thread, and `PieChart` for the end-of-session summary. **MongoDB Java Driver Documentation** (mongodb.github.io/mongo-java-driver/). `FindIterable` and `updateOne` for the strategy-write step. **OpenWeatherMap API Reference** (openweathermap.org/api). Current-conditions and five-day-forecast endpoints, both consumed by the WeatherService Façade. **ZXing GitHub Wiki** (github.com/zxing/zxing). QR encoding parameters for the session QR code.

4.4 SE 370 Codebase

The single most consulted resource was the team's own SE 370 submission (Final/lastSemester/SE370-2025-fall-team-02). Every refactor was made against the existing implementation, preserving public method signatures wherever possible so that untouched controllers continued to work. The SE 370 software design specification was used to confirm requirement coverage in Section 2.7 above.

5 Lessons Learned

5.1 Josh Clemens - Integration and Code Review

The biggest lesson for me this semester came from something I did not expect to matter as much as it did, having already helped build this app in SE 370. I went in thinking the head start would mostly save me time, I knew where the files lived and how the screens connected. What it really gave me was a feel for exactly where the original code was brittle. I remembered the late-night fixes from last semester, the parts I had patched over with a boolean here and an if-check there, the corners I had quietly hoped no one would test too hard. That memory turned the Observer, State, and Strategy refactor from a textbook exercise into something I could ground in our actual history with the codebase. What surprised me most was that the patterns did more than clean up the code, they made problems visible. Every pattern we wired in surfaced a real bug that had been hiding in the SE 370 build the whole time: sessions that did not clean themselves up when the instructor navigated away, attendance counts that did not match what was on screen, events firing twice without anyone noticing. None of those bugs were caused by the new patterns; they were already there. The patterns just gave us named events, logged transitions, and clearly-typed return values, and once those were in place the inconsistencies were hard to miss. By the

end of the semester I was reading every pull request through that lens instead of just checking whether a class diagram matched the slide, asking what evidence each pattern produced and what the next person could see now that they could not see before. Coming in with the context on this codebase made the patterns land much harder than they would have on a brand-new project, and the shift from thinking of design patterns as cleanup to thinking of them as a way to make a system tell you what is going on is the single most useful idea I'm taking out of SE 471.

5.2 Judah Fisher - Strategy

For this project I focused primarily on the implementation of the Strategy pattern. I started by reviewing some of the core concepts of Strategy and how to use it. It was my first time working with interfaces on a project and it was a good learning experience. Josh was super helpful throughout the process, he drew up UML diagrams and when I had any questions on how any components of WeatherGuard worked he was quick to answer them. I learned the value of having UML diagrams, in previous course work UML existed in my mind in a vacuum as it was usually made for homework or to present a project. Having a UML diagram and knowing how to quickly implement it was super helpful. Continuing with someone else's project was a challenge because there were so many components that I was unfamiliar with that I didn't want to break something. A big lesson I learned was to not be timid to mess around with the code and try new things. Through this I really learned the values of carefully having different branches to seriously mitigate any risk to the codebase.

5.3 Chima Ohaechesi - Observer

I focused on the Observer pattern for the most part for this project and really tried to learn the pattern and how it genuinely works inside a real application instead of just small class examples. At the beginning I understood the basics of the Observer from the lectures but implementing it into the WeatherGuard app made me really realize and understand how useful it is for organizing communication between different parts of the system. Josh had a lot of knowledge of the system from 370 so he really took charge on helping us out with the UML Diagrams, prior to the class I really didn't see the use of the uml diagrams. I mainly thought of it as extra work to look at. But looking on it now they really help me to visualize the relationships and apply the code implementation better, especially the design patterns are more practical tools than busy work, quite literally they make software easier to manage, expand and maintain instead of just concepts for class. This is the biggest thing and practically the most important thing I took away from the project.

5.4 Roger Karam - State

For this project, I focused primarily on implementing the State pattern, and it became one of the most practical design patterns I have worked with. Before this class, I understood the concept from lectures, but I had never applied it in a real application. Working on the WeatherGuard Attendance system showed me how useful the pattern is when an object's behavior changes depending on its current state. The biggest lesson I learned was how much cleaner the code becomes when each state is represented by its own class instead of using long chains of if and switch statements. In the original project, session behavior was controlled by checking flags to determine whether attendance was waiting to start, active, paused, or ended. As the project grew, this logic became harder to follow and debug. Using the State pattern made the behavior for each session state more organized and easier to maintain. Overall, this project changed how I think about the State pattern. I now see it as a practical way to organize complex logic and make software easier to understand, maintain, and debug. That is the most valuable lesson I am taking away from this project and from SE 471.